

Specifications for a C Numerical Test Suite for the Collatz Deterministic Defect Cocycle

Research Notes

June 9, 2026

Abstract

This document specifies an extensive C implementation for numerical experiments around the proposed deterministic defect cocycle for the accelerated Collatz/Syracuse map. The goal is not to prove Collatz. The goal is to understand whether persistent deterministic deviations from the annealed height law cast a finite spectral/coboundary shadow visible in tilted character sums, finite residue-height transfer operators, and their character-sector spectral gaps.

Contents

1	Purpose and Mathematical Target	3
2	Core Questions to Test	4
3	Implementation Language and Constraints	4
3.1	Required build profiles	4
3.2	Numerical precision	5
4	Module Architecture	5
5	Core Arithmetic Routines	5
5.1	Accelerated odd step	5
5.2	Deficit and height	6
6	Experiment A: Baseline Orbit and Height-Tail Census	6
6.1	Purpose	6
6.2	Input parameters	6
6.3	Outputs	6
6.4	Pre-registered expectation	7
7	Experiment B: Empirical Cramer-Tilt Law on Critical Excursions	7
7.1	Purpose	7
7.2	Measurements	7
7.3	Boundary layer tests	7
7.4	Outputs	7

8	Experiment C: Deterministic Defect Cocycle Sums	8
8.1	Purpose	8
8.2	Characters	8
8.3	Tilt grid	8
8.4	Window policies	8
8.5	Outputs	8
8.6	Pre-registered expectation	9
9	Experiment D: Post-Exit Taboo and Entropy-Space Repulsion	9
9.1	Purpose	9
9.2	Definitions	9
9.3	Outputs	9
10	Experiment E: Exact Residue-Height Operator and Character Gaps	9
10.1	Purpose	9
10.2	Finite state space	10
10.3	Edges	10
10.4	Weights	10
10.5	Spectral computation	10
10.6	Outputs	10
11	Experiment F: Annealed Fourier Prediction versus Exact Operator	11
11.1	Purpose	11
11.2	Outputs	11
12	Experiment G: Finite Determinant and Trace Shadows	11
12.1	Purpose	11
12.2	Methods	11
12.3	Outputs	11
13	Experiment H: Congruence Lemma Verification	12
13.1	Purpose	12
13.2	Outputs	12
14	Experiment I: Quenched Spectral-Shadow Search	12
14.1	Purpose	12
14.2	Window catalog	12
14.3	Outputs	13
14.4	Interpretation	13
15	Data Formats	13
16	Command-Line Interface	13
17	Performance Requirements	14
18	Validation and Sanity Checks	14
18.1	Arithmetic checks	14
18.2	Statistical checks	14
18.3	Operator checks	14

19 Pre-Registered Outcomes and Failure Modes	15
19.1 Expected if the defect-cocycle picture is right	15
19.2 Failure modes	15
20 Recommended Development Order	15
21 Final Deliverables	16

1 Purpose and Mathematical Target

The object under investigation is the pathwise finite-level detector

$$W_{n,\ell}(x; \chi, s) = \frac{1}{\ell} \sum_{i=n}^{n+\ell-1} \chi(x_i \bmod Q) 2^{s\Delta_{b_i}}, \quad \Delta_b = \log_2 3 - b,$$

where

$$x_{i+1} = S(x_i) = \frac{3x_i + 1}{2^{b_i}}, \quad b_i = v_2(3x_i + 1),$$

for odd positive integers. The character χ is a finite character on a residue modulus Q , and s is a Cramer/pressure tilt parameter. The most important values are

$$s = 0 \quad \text{and} \quad s = 1.$$

At $s = 0$, W is a residue Weyl sum. At $s = 1$, it probes the Cramer-tilted high-ascent regime, because

$$2^{\Delta_b} = 2^{\log_2 3 - b} = \frac{3}{2^b}.$$

The annealed valuation law is

$$p_b = 2^{-b}, \quad b \geq 1,$$

and the height transform is

$$M(s) = \sum_{b \geq 1} 2^{-b} 2^{s(\log_2 3 - b)} = \frac{3^s}{2^{1+s} - 1}.$$

The Cramer root is $M(1) = 1$, and the tilted law at the critical root is

$$p_b^{(1)} = 2^{-b} 2^{\log_2 3 - b} = 3 \cdot 4^{-b}.$$

The guiding hypothesis is not that nonzero W -sums immediately prove anything. Rather, they are proposed finite spectral shadows. The missing theorem would have the form

persistent bad deterministic behavior $\implies \exists(Q, L, \chi, s)$ with persistent nontrivial spectral shadow,
and H23a-type finite spectral gaps would then rule out that shadow.

2 Core Questions to Test

The numerical suite should test the following questions.

- Q1. **Does critical high-ascent behavior follow the Cramer-tilted law?** On high-ascent windows, do the empirical valuations satisfy

$$\hat{p}_b \approx 3 \cdot 4^{-b}?$$

- Q2. **Do high-ascent or sub-equilibrium windows carry residue resonance?** For relevant moduli Q , characters χ , and tilts s , are there windows with

$$|W_{n,\ell}(x; \chi, s)| \gg 0?$$

- Q3. **Is post-exit taboo visible as entropy-space repulsion?** After a completed critical excursion, is the next window separated from the critical tilted law by a uniform divergence

$$D(\hat{p}_{\text{post}} \| p^{(1)}) \geq c > 0?$$

- Q4. **Does the finite exact residue-height operator match the annealed Fourier shadow?** For finite (Q, L) , compare

$$\rho(\mathcal{L}_{s,Q,L,\chi})$$

against annealed model predictions and H23a-style gaps.

- Q5. **Can a bad cluster be seen before it disappears?** In any unusually long sub-equilibrium tail, high ascent, or near-critical excursion, does a finite character sector spike? If not, the proposed defect cocycle is missing the obstruction.

- Q6. **Are finite determinant/zeta shadows informative?** For finite operators, do traces

$$\text{tr}(\mathcal{L}_{s,Q,L}^n)$$

and determinants

$$\Xi_{Q,L}(z, s) = \det(1 - z\mathcal{L}_{s,Q,L})$$

show stable peripheral structure or only the trivial sector?

3 Implementation Language and Constraints

The implementation shall be in ISO C11, with optional compile-time use of GMP for ranges exceeding native integer arithmetic.

3.1 Required build profiles

FAST128 Uses `uint64_t` and `__uint128_t`. Intended for high-throughput scans where all intermediate values fit in 128 bits.

GMP Uses `mpz_t`. Intended for verification, very large seeds, and exact critical inequality checks beyond 128-bit range.

DEBUG Enables invariant checks, congruence verification, trace logging, and deterministic reproducibility checks.

3.2 Numerical precision

Floating point computations may use `long double` for real-valued logs, entropies, rates, and complex character sums. Exact integer predicates must not use floating point. In particular, the criticality condition

$$V^2 \leq 2^A P$$

must be checked using `__uint128_t` or GMP.

4 Module Architecture

Module	Responsibility
<code>collatz_core.c/h</code>	Accelerated odd Syracuse step; valuation computation; overflow-safe update; exact affine iterate accumulator when requested.
<code>orbit_scan.c/h</code>	Streams orbit segments, windows, stopping conditions, cycle detection, max height, deficit, and critical events.
<code>characters.c/h</code>	Finite residue characters modulo Q , including additive characters $e^{2\pi i k a/Q}$, multiplicative unit characters where applicable, and product characters for mixed moduli.
<code>defect_sums.c/h</code>	Computes prefix and sliding-window defect sums $W_{n,\ell}(x; \chi, s)$.
<code>entropy_stats.c/h</code>	Empirical valuation laws, KL divergences, entropies, Cramer tilt comparisons, boundary-layer statistics.
<code>critical_events.c/h</code>	Detects high ascents, C_A -critical launch floors, exits, post-exit windows, and taboo candidates.
<code>residue_height_graph.c/h</code>	Builds finite exact residue-height graphs $\mathcal{X}_{Q,L}$, admissible edges, killing boundaries, and weights.
<code>sparse_matrix.c/h</code>	CSR sparse matrices, complex weights, power iteration, Arnoldi/Lanczos-lite where applicable.
<code>spectral_tests.c/h</code>	Spectral radius estimates of $\mathcal{L}_{s,Q,L,\chi}$, sector gaps, finite determinant/traces for small matrices.
<code>io_csv.c/h</code>	Streaming CSV/JSONL writers, metadata headers, reproducibility manifest.
<code>config.c/h</code>	CLI parsing, experiment configuration, seed ranges, moduli, tilts, windows, output paths.

5 Core Arithmetic Routines

5.1 Accelerated odd step

For an odd input x , compute

$$y = 3x + 1, \quad b = v_2(y), \quad S(x) = y/2^b.$$

```
typedef struct {
    uint64_t next;
    uint32_t b;
    int overflow;
```

```

} syr_step64_t;

syr_step64_t syracuse_step_u64(uint64_t x);

```

For FAST128, compute $3x + 1$ in `__uint128_t`, then count trailing zeros. For GMP, use `mpz_mul_ui`, `mpz_add_ui`, `mpz_scan1`, and `mpz_tdiv_q_2exp`.

5.2 Deficit and height

Maintain

$$D_m = \sum_{i < m} b_i, \quad E_m = D_m - m \log_2 3,$$

and

$$H_m = \log_2 x_m - \log_2 x_0.$$

For exact integer comparisons, also maintain peak values where feasible. Floating logs are used only for diagnostics.

6 Experiment A: Baseline Orbit and Height-Tail Census

6.1 Purpose

Confirm the large-deviation picture

$$\Pr(h \geq H) \asymp 2^{-H},$$

and the octave-uniform count

$$\#(C_A \cap [2^N, 2^{N+1})) = O_A(1)$$

for fixed aperture A , especially $A = 6$.

6.2 Input parameters

- Octave range $N_{\min} \leq N \leq N_{\max}$.
- Seed policy: all odd seeds in octave, arithmetic progression, random sample, or residue-filtered sample.
- Apertures $A \in \{4, 5, 6, 7, 8\}$ by default.
- Maximum steps per seed.

6.3 Outputs

Write `height_tail.csv` with columns:

```

N, seeds_tested, A, critical_count, max_height, mean_height,
tail_count_H0, tail_count_H1, ..., runtime_seconds

```

Write `critical_events.csv` with columns:

```

seed, octave_N, A, launch_V, peak_P, exit_X, start_step,
end_step, word_length, D, height_log2, amin_exact_num,
amin_exact_den_power2, overshoot, status

```

6.4 Pre-registered expectation

For fixed A , counts per octave should remain bounded or grow slowly. Height tails should fit slope -1 in base-2 log scale:

$$\log_2 \Pr(h \geq H) \approx -H + C.$$

7 Experiment B: Empirical Cramer-Tilt Law on Critical Excursions

7.1 Purpose

Test whether the bulk valuation law of critical excursions converges to

$$p_b^{(1)} = 3 \cdot 4^{-b}.$$

7.2 Measurements

For each critical excursion word $w = (b_0, \dots, b_{\ell-1})$, compute:

$$\hat{p}_b(w) = \frac{1}{\ell} \#\{i : b_i = b\},$$

$$D(\hat{p} \| p), \quad D(\hat{p} \| p^{(1)}),$$

where

$$p_b = 2^{-b}, \quad p_b^{(1)} = 3 \cdot 4^{-b}.$$

Also compute the entropy per consumed binary digit

$$d(w) = \frac{H(\hat{p})}{\sum_b b \hat{p}_b}.$$

The expected limiting critical bulk dimension is

$$H_2(3/4) = 0.811278 \dots$$

under the simplified geometric-tilt model.

7.3 Boundary layer tests

For each critical word, compute empirical laws on:

- first r symbols,
- middle bulk excluding first and last r ,
- last r symbols,

for $r \in \{4, 8, 16, 32\}$. This tests whether the earlier spurious dimension estimates are boundary-layer effects.

7.4 Outputs

Write `tilt_law.csv` with columns:

```
event_id, seed, N, A, segment_type, r, length,
p1_hat, p2_hat, p3_hat, p4_hat, tail_b_ge_5,
KL_to_p, KL_to_p1, entropy, mean_b, entropy_per_digit
```

8 Experiment C: Deterministic Defect Cocycle Sums

8.1 Purpose

Compute pathwise finite spectral shadows

$$W_{n,\ell}(x; \chi, s) = \frac{1}{\ell} \sum_{i=n}^{n+\ell-1} \chi(x_i \bmod Q) 2^{s(\log_2 3 - b_i)}.$$

8.2 Characters

Support the following character families:

1. Additive characters modulo Q :

$$\chi_k(a) = \exp(2\pi i k a / Q), \quad 1 \leq k < Q.$$

2. Unit multiplicative characters for $Q = 3^r$ and odd unit groups where feasible.
3. Product characters for mixed $Q = 2^a 3^r$, implemented via CRT factorization.

For initial work, additive characters are sufficient and simplest.

8.3 Tilt grid

Default tilt values:

$$s \in \{0, 1/4, 1/2, 3/4, 1, 5/4, 3/2\}.$$

Always include $s = 0$ and $s = 1$.

8.4 Window policies

Compute both prefix and sliding-window sums:

$$W_n(x; \chi, s) = W_{0,n}(x; \chi, s),$$
$$W_{n,\ell}(x; \chi, s), \quad \ell \in \{16, 32, 64, 128, 256, 512, 1024, 2048\}.$$

8.5 Outputs

Write `defect_windows.csv`:

```
seed, start_step, length, Q, char_type, char_index, s,  
real_W, imag_W, abs_W, mean_delta, mean_b,  
segment_label, event_id
```

Segment labels:

```
ordinary, high_ascent, critical_C_A, post_exit, sub_equilibrium_tail,  
cycle_neighborhood, unknown
```


8.6 Pre-registered expectation

Ordinary windows should show decay consistent with finite-character equidistribution. Critical windows may show the Cramer tilt in b -statistics but should not show persistent nontrivial residue resonance if the H23a picture is correct. A persistent large $|W|$ during bad clusters would support the finite spectral-shadow principle.

9 Experiment D: Post-Exit Taboo and Entropy-Space Repulsion

9.1 Purpose

Test whether completed critical excursions are followed by windows that are separated from the critical tilted law, explaining why two 6-critical excursions fail to concatenate.

9.2 Definitions

For a completed critical event e , define the post-exit window of length ℓ :

$$w_{\text{post}}(e, \ell) = (b_{t_e}, \dots, b_{t_e + \ell - 1}),$$

where t_e is the first step after the exit.

Compute:

$$D(\hat{p}_{\text{post}} \| p^{(1)}), \quad M_{\text{post}}(1) = \sum_b \hat{p}_{\text{post}}(b) 2^{\log_2 3 - b}.$$

If

$$M_{\text{post}}(1) < 1$$

with a stable margin, then post-exit behavior is subcritical relative to the high-ascent tilt.

9.3 Outputs

Write `post_exit.csv`:

```
event_id, seed, A, exit_step, post_length,
KL_post_to_p, KL_post_to_p1, M_post_1,
p1_post, p2_post, p3_post, mean_b_post,
next_critical_flag, next_critical_A, gap_steps
```

10 Experiment E: Exact Residue-Height Operator and Character Gaps

10.1 Purpose

Build finite exact killed residue-height operators $\mathcal{L}_{s,Q,L,\chi}$ and estimate spectral gaps:

$$\rho(\mathcal{L}_{s,Q,L,\chi}) \leq (1 - \kappa_Q) \rho(\mathcal{L}_{s,Q,L,0}).$$

10.2 Finite state space

Let

$$\mathcal{X}_{Q,L} = \{(a, z) : a \in \Omega_Q, z \in \{0, 1, \dots, L\}\}.$$

The coordinate z may be a scaled integer approximation to deficit/height. The scale factor R should be configurable, so that

$$z \approx R \cdot \text{height}.$$

10.3 Edges

For each state and valuation $b \leq B_{\max}$, construct an edge if admissible:

$$a' \equiv 3^{-1}(2^b a - 1) \pmod{Q}.$$

Update height by

$$z' = z + R(\log_2 3 - b)$$

with deterministic rounding rule. Edges with $z' \notin [0, L]$ are killed.

The truncation error from ignoring $b > B_{\max}$ must be reported:

$$\sum_{b > B_{\max}} 2^{-b} = 2^{-B_{\max}}.$$

10.4 Weights

Default annealed edge weight:

$$w_s(b) = 2^{-b} 2^{s(\log_2 3 - b)}.$$

Exact finite-fiber weights should be supported as a separate mode.

For character sector χ , multiply by the character phase

$$\chi(a \rightarrow a')$$

according to the chosen character convention. The convention must be recorded in metadata.

10.5 Spectral computation

Use sparse power iteration for $\chi = 1$. For complex nontrivial sectors, estimate spectral radius by:

- power iteration on A^*A for singular radius as an upper bound,
- Arnoldi iteration for eigenvalue estimates,
- repeated random starts to avoid missing peripheral modes.

10.6 Outputs

Write `spectral_gaps.csv`:

```
Q, L, R, Bmax, s, char_type, char_index,
rho_est, rho0_est, gap_ratio, iterations, residual,
truncation_tail, matrix_nnz, matrix_dim
```

11 Experiment F: Annealed Fourier Prediction versus Exact Operator

11.1 Purpose

Compare exact finite operator gaps with the simple i.i.d. valuation Fourier transform

$$\hat{p}(\xi) = \sum_{b \geq 1} 2^{-b} e(b\xi) = \frac{e(\xi)}{2 - e(\xi)}.$$

Thus

$$|\hat{p}(\xi)| = \frac{1}{\sqrt{5 - 4 \cos(2\pi\xi)}} < 1$$

for $\xi \notin \mathbb{Z}$.

11.2 Outputs

Write `fourier_comparison.csv`:

```
Q, character_index, xi, annealed_abs_hat_p,  
exact_gap_ratio_s0, exact_gap_ratio_s1, deviation
```

12 Experiment G: Finite Determinant and Trace Shadows

12.1 Purpose

For small (Q, L) , compute finite traces

$$\text{tr}(\mathcal{L}_{s,Q,L,\chi}^n)$$

and determinants

$$\Xi_{Q,L,\chi}(z, s) = \det(1 - z\mathcal{L}_{s,Q,L,\chi}).$$

This tests whether closed-path data show unexpected peripheral structure.

12.2 Methods

For small dense matrices, use direct LU decomposition to compute determinants at selected z . For sparse matrices, estimate traces by explicit sparse multiplication for low powers or Hutchinson trace estimators.

12.3 Outputs

Write `trace_determinant.csv`:

```
Q, L, s, char_index, n, trace_real, trace_imag,  
trace_abs, determinant_real, determinant_imag, z_real, z_imag
```

13 Experiment H: Congruence Lemma Verification

13.1 Purpose

Verify the sliding-window congruence formula modulo 3^r . Starting from

$$x_n = \frac{3^n x_0 + A_n}{2^{B_n}}, \quad A_n = \sum_{j=0}^{n-1} 3^{n-1-j} 2^{B_j},$$

with

$$B_n = \sum_{i < n} b_i,$$

then for $n \geq r$, modulo 3^r , the $3^n x_0$ term vanishes and only the last r terms survive:

$$x_n \equiv \sum_{k=1}^r 3^{k-1} 2^{-(b_{n-k} + \dots + b_{n-1})} \pmod{3^r}.$$

13.2 Outputs

Write `congruence_checks.csv`:

```
seed, n, r, modulus, direct_residue, formula_residue,
match_flag, Bn_mod_phi, status
```

Any mismatch is a bug unless overflow or a documented convention error is present.

14 Experiment I: Quenched Spectral-Shadow Search

14.1 Purpose

This is the central integration experiment. Given a catalog of unusual deterministic windows, determine whether they cast a finite spectral shadow.

14.2 Window catalog

Collect windows from:

- largest positive height excursions,
- deepest sub-equilibrium deficit tails,
- completed C_A -critical excursions,
- post-exit windows,
- record-setting values of $|E_m|$,
- windows with extreme KL divergence from p ,
- windows close to $p^{(1)}$.

For each window, compute all configured $W_{n,\ell}(x; \chi, s)$. A window is flagged as shadow-bearing if

$$\max_{Q, \chi, s} |W_{n,\ell}(x; \chi, s)| \geq \delta$$

for configurable δ , default $\delta = 0.1$.

14.3 Outputs

Write `shadow_search.csv`:

```
window_id, seed, start_step, length, label,  
max_abs_W, argmax_Q, argmax_char, argmax_s,  
KL_to_p, KL_to_p1, mean_delta, max_height,  
shadow_flag
```

14.4 Interpretation

If bad/critical windows repeatedly have no finite spectral shadow across growing (Q, L) , then the proposed defect cocycle is probably not the right detector. If they do have shadows, the next task is to relate those shadows to finite operator gaps.

15 Data Formats

All outputs shall be append-only CSV with a metadata header. Each file starts with commented lines:

```
# experiment=defect_windows  
# git_commit=<commit hash if available>  
# compiler=<compiler and version>  
# build_profile=FAST128  
# command=<full command line>  
# timestamp_utc=<ISO8601>  
# random_seed=<if applicable>
```

Large runs may additionally write JSONL summaries.

16 Command-Line Interface

The executable should support subcommands:

```
collatz_tests scan-height [options]  
collatz_tests tilt-law [options]  
collatz_tests defect-windows [options]  
collatz_tests post-exit [options]  
collatz_tests spectral-gaps [options]  
collatz_tests fourier-compare [options]  
collatz_tests trace-det [options]  
collatz_tests congruence-check [options]  
collatz_tests shadow-search [options]
```

Common options:

```
--start N  
--end N  
--octave-min N  
--octave-max N  
--max-steps N  
--windows 16,32,64,128,256,512,1024  
--moduli 8,16,27,81,243,729
```

```

--tilts 0,0.5,1,1.5
--apertures 4,5,6,7,8
--output-dir path
--threads N
--profile FAST128|GMP
--debug-checks

```

17 Performance Requirements

1. Orbit scanning must be streaming and must not store full trajectories unless explicitly requested.
2. Sliding windows should use ring buffers for b_i , residues, and precomputed weights.
3. Character values for fixed (Q, k) should be precomputed in arrays of complex `long double` values.
4. Sparse matrices should use CSR format.
5. Long scans should support checkpointing every fixed number of seeds.
6. Parallel scans may use OpenMP, but output writing must be deterministic or partitioned by thread into mergeable files.

18 Validation and Sanity Checks

18.1 Arithmetic checks

- Confirm every accelerated step returns an odd integer.
- Confirm $3x + 1 = 2^b x'$ exactly.
- Confirm no overflow in FAST128; otherwise set overflow flag and skip or rerun in GMP.
- Confirm criticality inequalities by exact integer arithmetic.

18.2 Statistical checks

For random odd seeds modulo large powers of two, empirical b -frequencies should satisfy

$$\hat{p}_b \approx 2^{-b}.$$

For critical high-ascent bulk windows, empirical frequencies should approach

$$\hat{p}_b \approx 3 \cdot 4^{-b}.$$

18.3 Operator checks

For $\chi = 1$, spectral radius should dominate nontrivial sectors. For trivial small cases, direct dense eigenvalue computations should match sparse power estimates.

19 Pre-Registered Outcomes and Failure Modes

19.1 Expected if the defect-cocycle picture is right

1. Critical windows follow the Cramer tilt $p_b^{(1)} = 3 \cdot 4^{-b}$ in the bulk.
2. Post-exit windows are separated from $p^{(1)}$ or have $M_{\text{post}}(1) < 1$.
3. Nontrivial character sums decay on ordinary windows.
4. Any unusually persistent bad window casts a finite W -shadow at some (Q, χ, s) .
5. Exact finite character sectors have stable spectral gaps consistent with H23a.

19.2 Failure modes

1. Bad windows exist but no finite W -shadow appears. Then the defect cocycle is not detecting the obstruction.
2. Large W -shadows appear in ordinary windows. Then the character family or normalization is too sensitive or biased.
3. Critical windows do not approach $p^{(1)}$. Then the current Cramer-tilt model is incomplete.
4. Post-exit windows can approach $p^{(1)}$ with no penalty. Then the taboo mechanism is not real or requires another coordinate.
5. Exact finite spectral gaps deteriorate with L or Q . Then H23a-style finite obstruction exclusion is insufficient.

20 Recommended Development Order

1. Implement `collatz_core`, `orbit_scan`, and exact critical event detection.
2. Implement Experiment A and B to reproduce height-tail and Cramer-tilt law.
3. Implement character precomputation and Experiment C defect windows.
4. Implement post-exit Experiment D.
5. Implement finite residue-height graph construction and Experiment E.
6. Implement Fourier comparison Experiment F.
7. Implement determinant/trace Experiment G only after E is stable.
8. Integrate everything in Experiment I shadow search.

21 Final Deliverables

The C project should produce:

1. A command-line executable `collatz_tests`.
2. A reproducible configuration file format.
3. CSV/JSONL outputs for every experiment.
4. A validation report comparing known annealed laws to empirical results.
5. A shadow-search report ranking windows by maximal finite defect signal.
6. A spectral-gap report for finite residue-height operators.

The purpose of the suite is not to prove the finite spectral-shadow principle. Its purpose is to determine whether that principle is numerically plausible, what finite moduli and tilts see the obstruction most clearly, and whether the deterministic defect cocycle $W_{n,\ell}(x; \chi, s)$ is the right finite object to pursue.